Classification Models

Prog 3- KNN

```
import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

# Training data: Study Hours, Sleep Hours

X = [[8,8], [7,7], [9,8], [1,4],
     [2,5], [3,4], [8,6], [2,3]]

y = ["Pass", "Pass", "Pass", "Fail",
     "Fail", "Fail", "Pass", "Fail"]

# Train model

model = KNeighborsClassifier(n_neighbors=3)

model.fit(X, y)

# User input

study = float(input("Enter study hours: "))

sleep = float(input("Enter sleep hours: "))

# Prediction

prediction = model.predict([[study, sleep]])[0]

print("Student Result:", prediction.upper())

# Graph

for i in range(len(X)):

    plt.scatter(X[i][0], X[i][1],
                color="green" if y[i] == "Pass" else "red")

plt.scatter(study, sleep, color="blue", marker="*", s=250)

plt.xlabel("Study Hours")

plt.ylabel("Sleep Hours")
```

```
plt.title("KNN Classification - Pass or Fail")

plt.show()
import matplotlib.pyplot as plt
```

Prog 5 - Naïve Bayes

```
from sklearn.naive_bayes import GaussianNB

# Training data: Study Hours, Sleep Hours

X = [[8,8], [7,7], [9,8], [1,4],
     [2,5], [3,4], [8,6], [2,3]]

y = ["Pass", "Pass", "Pass", "Fail",
     "Fail", "Fail", "Pass", "Fail"]

# Train model

model = GaussianNB()

model.fit(X, y)

# User input

study = float(input("Enter study hours: "))

sleep = float(input("Enter sleep hours: "))

# Prediction

prediction = model.predict([[study, sleep]])[0]

print("Student Result:", prediction.upper())

# Graph

for i in range(len(X)):

    plt.scatter(X[i][0], X[i][1],
                color="green" if y[i] == "Pass" else "red")

plt.scatter(study, sleep, color="blue", marker="*", s=250)

plt.xlabel("Study Hours")
```

```
plt.ylabel("Sleep Hours")

plt.title("Naive Bayes Classification - Pass or Fail")

plt.show()
```

Prog 6 – Decision Tree Classifier

```
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier

# Training data: Study Hours, Sleep Hours

X = [[8,8], [7,7], [9,8], [1,4],
     [2,5], [3,4], [8,6], [2,3]]

y = ["Pass", "Pass", "Pass", "Fail",
     "Fail", "Fail", "Pass", "Fail"]

# Train model

model = DecisionTreeClassifier()

model.fit(X, y)

# User input

study = float(input("Enter study hours: "))

sleep = float(input("Enter sleep hours: "))

# Prediction

prediction = model.predict([[study, sleep]])[0]

print("Student Result:", prediction.upper())

# Graph

for i in range(len(X)):

    plt.scatter(X[i][0], X[i][1],
                color="green" if y[i] == "Pass" else "red")
```

```
plt.scatter(study, sleep, color="blue", marker="*", s=250)

plt.xlabel("Study Hours")

plt.ylabel("Sleep Hours")

plt.title("Decision Tree Classification - Pass or Fail")

plt.show()
```

Prog 8 – Random Forest

```
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier

# Training data: Study Hours, Sleep Hours

X = [[8,8], [7,7], [9,8], [1,4],
    [2,5], [3,4], [8,6], [2,3]]

y = ["Pass", "Pass", "Pass", "Fail",
    "Fail", "Fail", "Pass", "Fail"]

# Train model

model = RandomForestClassifier(n_estimators=10, random_state=42)

model.fit(X, y)

# User input

study = float(input("Enter study hours: "))

sleep = float(input("Enter sleep hours: "))

# Prediction

prediction = model.predict([[study, sleep]])[0]

print("Student Result:", prediction.upper())

# Graph

for i in range(len(X)):
```

```
    plt.scatter(X[i][0], X[i][1],
                color="green" if y[i] == "Pass" else "red")

plt.scatter(study, sleep, color="blue", marker="*", s=250)

plt.xlabel("Study Hours")

plt.ylabel("Sleep Hours")

plt.title("Random Forest Classification - Pass or Fail")

plt.show()
```

Regression Models

Prog 4 – Linear Regression

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Training data (Seen Data)
X = [[500], [700], [900], [1100], [1300], [1500]]
y = [20, 28, 36, 44, 52, 60]  # Price in Lakhs

# Create and train the model
model = LinearRegression()
model.fit(X, y)

# User input (Unseen Data)
area = float(input("Enter house area in sq. ft.: "))

# Predict house price
prediction = model.predict([[area]])

print("Predicted House Price:", round(prediction[0], 2), "Lakhs")

# Graph
plt.scatter(X, y, color="green", label="Training Data")
plt.scatter(area, prediction[0], color="red", marker="*", s=150, label="New House")
plt.plot(X, model.predict(X), color="blue", label="Regression Line")

plt.xlabel("House Area (sq. ft.)")
plt.ylabel("House Price (Lakhs)")
plt.title("Linear Regression - House Price Prediction")
plt.legend()
plt.show()
```

Prog 9- Multiple Regression

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Training data (Seen Data)
# Area, Bedrooms, Parking (1 = Yes, 0 = No)
X = [
    [500, 1, 0],
    [700, 2, 1],
    [900, 2, 0],
    [1100, 3, 1],
    [1300, 3, 1],
    [1500, 4, 1]
]

y = [20, 30, 32, 48, 55, 65]  # Price in Lakhs

# Create and train the model
model = LinearRegression()
model.fit(X, y)

# User input (Unseen Data)
area = float(input("Enter house area in sq. ft.: "))
bedrooms = int(input("Enter number of bedrooms: "))
parking = int(input("Parking available? (1 = Yes, 0 = No): "))

# Predict house price
prediction = model.predict([[area, bedrooms, parking]])

print("Predicted House Price:", round(prediction[0], 2), "Lakhs")

# Graph (Area vs Price)
plt.scatter([row[0] for row in X], y, color="green", label="Training Data")
plt.scatter(area, prediction[0], color="red", marker="*", s=150, label="New House")

plt.xlabel("House Area (sq. ft.)")
plt.ylabel("House Price (Lakhs)")
plt.title("Multiple Linear Regression - House Price Prediction")
plt.legend()
plt.show()
```